Lecture Notes: TwinCAT 3

Extending Functionality with TwinCAT Functions

March 14, 2026

Contents

1 Introduction to TwinCAT Functions **2**

2 Categories of TwinCAT Functions **2**
- 2.1 TF1xxx (System) . . . 2
- 2.2 TF2xxx (HMI) . . . 2
- 2.3 TF3xxx (Measurement) . . . 2
- 2.4 TF4xxx (Controller) . . . 2
- 2.5 TF5xxx (Motion Control) . . . 2
- 2.6 TF6xxx (Connectivity) . . . 3
- 2.7 TF7xxx (Vision) . . . 3
- 2.8 TF8xxx (Industry Specific) . . . 3

3 Architecture of a Connectivity Function (Modbus TCP) **3**
- 3.1 Installation . . . 3
- 3.2 The Process . . . 3
- 3.3 Communication Bridge . . . 3
- 3.4 Asynchronous Execution . . . 3

4 Programming and Licensing **4**
- 4.1 Libraries . . . 4
- 4.2 Licensing . . . 4
- 4.3 Modbus Protocol Basics . . . 4

5 Implementation: State Machine Logic **4**
- 5.1 State Machine Enumeration . . . 4
- 5.2 Modbus Read Coils Implementation . . . 5

6 Practical Workflow: Reading and Writing **7**
- 6.1 Modbus Write Register Implementation . . . 7

7 Key Considerations **10**
- 7.1 Virtual Machines . . . 10
- 7.2 Error Handling . . . 10
- 7.3 Port 502 . . . 10

Quick Reference Card **11**

1 Introduction to TwinCAT Functions

While the base installation of TwinCAT 3 (XAE) covers the majority of industrial automation needs, certain specialized tasks require extending the system's core capabilities. This is achieved through **TwinCAT Functions**, an ecosystem of extensions categorized by their specific application area.

> ℹ **Note:** TwinCAT Functions are add-on modules that extend the base TwinCAT functionality. They are installed separately and require their own licenses (though trial licenses are available for testing).

2 Categories of TwinCAT Functions

TwinCAT functions are organized into several major series:

Series	Category	Examples
TF1xxx	System	Matlab/Simulink, UML editors
TF2xxx	HMI	HTML5, CSS, JavaScript interfaces
TF3xxx	Measurement	TwinCAT Scope, power measurement
TF4xxx	Controller	PID controllers, filters, speech commands
TF5xxx	Motion Control	Axis control, multi-axis synchronization
TF6xxx	Connectivity	JSON, OPC-UA, Modbus TCP, PROFINET, FTP
TF7xxx	Vision	Camera integration, quality inspection
TF8xxx	Industry Specific	Building automation, wind power

Table 1: TwinCAT Function Categories

2.1 TF1xxx (System)

Extensions for the PLC runtime, such as running Matlab/Simulink models or using UML editors.

2.2 TF2xxx (HMI)

Tools for creating modern operator interfaces using HTML5, CSS, and JavaScript.

2.3 TF3xxx (Measurement)

Tools for analytics, power measurement, and the TwinCAT Scope for process analysis.

2.4 TF4xxx (Controller)

Includes PID controllers, various filters, and even speech-to-PLC command functions.

2.5 TF5xxx (Motion Control)

A massive category covering all aspects of motion, from simple axis control to complex multi-axis synchronization.

2.6 TF6xxx (Connectivity)

Enables the PLC to communicate with external systems via protocols like:

- JSON (TF6020)
- OPC-UA (TF6100)
- Modbus TCP (TF6250)
- PROFINET (TF6270)
- FTP (TF6300)

2.7 TF7xxx (Vision)

Integrates camera support directly into the TwinCAT environment for quality inspection.

2.8 TF8xxx (Industry Specific)

Specialized frameworks for industries like building automation or wind power.

3 Architecture of a Connectivity Function (Modbus TCP)

Using a function like **TF6250 (Modbus TCP)** involves a specific architecture to bridge the gap between real-time control and standard networking.

3.1 Installation

The function is a separate executable downloaded from the Beckhoff website and installed on the PLC runtime (XAR).

3.2 The Process

Installation creates a background process in the Windows operating system (user-land).

3.3 Communication Bridge

The real-time TwinCAT kernel communicates with this Windows process (likely via ADS). The Windows process then utilizes the standard network stack to talk to external Modbus devices.

3.4 Asynchronous Execution

Because the networking happens in non-deterministic Windows space, the communication is **asynchronous** to the real-time PLC cycle.

PLC Code (Real-time)	ADS	**Windows Process** (TF6250 Service)	TCP/IP
TwinCAT Kernel		Windows User Space	

Modbus Device (External PLC, Sensor, etc.)

Figure 1: TF6250 Modbus TCP Architecture

4 Programming and Licensing

4.1 Libraries

To use the function, you must add a reference to the specific library (e.g., `Tc2_ModbusTCP`) in your PLC project to access its function blocks.

4.2 Licensing

These extensions are not free. They require a specific license (e.g., TF6250) added in the "Manage Licenses" tab. For testing, 7-day trial licenses can be generated.

4.3 Modbus Protocol Basics

Modbus is a primitive protocol from the 1970s that works with bits (coils) and 16-bit values (registers) rather than named variables.

Modbus Type	Data Type	Access	Function Codes
Coils	Single bit (BOOL)	Read/Write	01, 05, 15
Discrete Inputs	Single bit (BOOL)	Read only	02
Holding Registers	16-bit (WORD)	Read/Write	03, 06, 16
Input Registers	16-bit (WORD)	Read only	04

Table 2: Modbus Data Types

⚠ **Warning:** ADS Error **1828 (0x724)** specifically indicates a missing license for the TwinCAT function. Ensure you have activated the appropriate license or trial before testing.

5 Implementation: State Machine Logic

Because Modbus communication is asynchronous, you cannot simply call a function block and expect an immediate result. You must implement a state machine to manage the request.

5.1 State Machine Enumeration

```
// Local enumeration for managing asynchronous Modbus read operations
// Using local enums prevents namespace pollution and makes code self-documenting
// The {attribute 'strict'} ensures type safety

FUNCTION_BLOCK FB_ModbusReader
VAR
    // Local enumeration for state machine states
    // Defines the asynchronous communication flow
    {attribute 'strict'}
    E_ReadState : (
        WAIT,        // Idle state - waiting for trigger command
        TRIGGER,     // Send the Modbus request (set bExecute := TRUE)
        BUSY         // Wait for response (monitor bBusy and bError)
    );
```

```

    // Current state variable
    eReadState : E_ReadState := E_ReadState.WAIT;

    // Trigger to start a read operation
    bStartRead : BOOL;

    // Status outputs
    bReadComplete : BOOL;
    bReadError    : BOOL;
    nErrorCode    : UDINT;
END_VAR
```

```
// Alternative: Separate enumeration for write operations
// You may need different states for different operation types

VAR
    // Enumeration for write state machine
    {attribute 'strict'}
    E_WriteState : (
        IDLE,            // Waiting for write command
        SEND_REQUEST,    // Trigger the write operation
        WAIT_RESPONSE,   // Wait for confirmation
        COMPLETE,        // Write successful
        ERROR            // Write failed
    );

    eWriteState : E_WriteState := E_WriteState.IDLE;
END_VAR
```

Tip: Using local enumerations within function blocks (rather than global types) keeps your code organized and prevents naming conflicts when multiple function blocks need similar state definitions.

5.2 Modbus Read Coils Implementation

```
// Modbus TCP Read Coils Implementation
// Reads digital inputs (bits) from a remote Modbus device
// Library required: Tc2_ModbusTCP

FUNCTION_BLOCK FB_ModbusCoilReader
VAR
    // State machine enumeration
    {attribute 'strict'}
    E_ReadState : (
        WAIT,        // Idle - waiting for trigger
        TRIGGER,     // Start the read operation
        READ         // Wait for completion (BUSY state)
    );

    eReadState : E_ReadState := E_ReadState.WAIT;

    // Modbus Read Coils function block instance
    // From Tc2_ModbusTCP library
    fbReadCoils : FB_MBReadCoils;
```

```

    // Local buffer to store the read coil data
    // Each BYTE can hold 8 coil values (bits)
    // Size depends on nQuantity: need nQuantity/8 bytes (rounded up)
    aCoilBuffer : ARRAY[0..9] OF BYTE;  // Buffer for up to 80 coils

    // Configuration parameters
    sDeviceIP    : STRING(15) := '192.168.1.100';  // Target device IP
    nTCPPort     : UINT := 502;                     // Standard Modbus port
    nUnitID      : BYTE := 255;                     // Modbus unit ID (255 = broadcast)
    nStartAddr   : UINT := 0;                       // Starting coil address
    nNumCoils    : UINT := 8;                       // Number of coils to read

    // Control and status
    bExecuteRead    : BOOL;            // Trigger to start read
    bReadDone       : BOOL;            // Read completed successfully
    bReadError      : BOOL;            // Read failed
    nErrorID        : UDINT;           // Error code if failed

    // Extracted coil values (for convenience)
    bCoil0          : BOOL;            // First coil value
    bCoil1          : BOOL;            // Second coil value
END_VAR
```

```
// State machine implementation for reading coils
CASE eReadState OF

    E_ReadState.WAIT:
        // Idle state - waiting for trigger
        bReadDone := FALSE;
        bReadError := FALSE;

        IF bExecuteRead THEN
            bExecuteRead := FALSE;  // Reset trigger
            eReadState := E_ReadState.TRIGGER;
        END_IF

    E_ReadState.TRIGGER:
        // Start the Modbus read operation
        // Configure and trigger the function block
        fbReadCoils(
            sIPAddr      := sDeviceIP,           // Target device IP address
            nTCPPort     := nTCPPort,            // TCP port (502 standard)
            nUnitID      := nUnitID,             // Modbus unit identifier
            nQuantity    := nNumCoils,           // Number of coils to read
            nMBAddr      := nStartAddr,          // Starting Modbus address
            cbLength     := SIZEOF(aCoilBuffer), // Buffer size
            pDestAddr    := ADR(aCoilBuffer),    // Pointer to destination buffer
```

```
            bExecute     := TRUE,                    // TRIGGER the operation
            tTimeout     := T#5S                     // 5 second timeout
        );

        eReadState := E_ReadState.READ;

    E_ReadState.READ:
        // Wait for operation to complete
        // Must continue calling FB with bExecute := FALSE
        fbReadCoils(bExecute := FALSE);

        // Check if operation is complete
        IF NOT fbReadCoils.bBusy THEN
            IF fbReadCoils.bError THEN
                // Read failed
                bReadError := TRUE;
                nErrorID := fbReadCoils.nErrId;
            ELSE
                // Read successful - extract coil values
                bReadDone := TRUE;

                // Extract individual bits from buffer
                // Coil 0 is bit 0 of byte 0, etc.
                bCoil0 := (aCoilBuffer[0] AND 16#01) > 0;
                bCoil1 := (aCoilBuffer[0] AND 16#02) > 0;
            END_IF

            eReadState := E_ReadState.WAIT;
        END_IF

END_CASE
```

Note: The `ADR()` function returns a pointer to the variable. This is required because the Modbus function blocks write data directly to memory at the specified address. The `cbLength` parameter tells the function block how much space is available in the buffer.

6 Practical Workflow: Reading and Writing

1. **Wait State:** The program remains idle until a trigger is received.
2. **Trigger State:** Set the `execute` flag of the Modbus function block to **TRUE** to send the command to the Windows process.
3. **Busy/Read State:** Set `execute` back to **FALSE** and monitor the `bBusy` and `bError` outputs.
4. **Completion:** Once `bBusy` is false, check `bError`. If successful, the data is available in your local buffer.

6.1 Modbus Write Register Implementation

WAIT	$\xrightarrow{\text{Trigger}}$	**TRIGGER**	$\xrightarrow{\text{Immediate}}$
bExecute = FALSE		bExecute = TRUE	

$\xrightarrow{\text{Immediate}}$	**BUSY/READ**	$\xrightarrow{\text{!bBusy}}$
bExecute = FALSE, monitor bBusy		Complete or Error

Figure 2: Asynchronous Communication State Flow

```
// Modbus TCP Write Single Register Implementation
// Writes a 16-bit value to a remote Modbus device
// Library required: Tc2_ModbusTCP

FUNCTION_BLOCK FB_ModbusRegisterWriter
VAR
    // State machine enumeration
    {attribute 'strict'}
    E_WriteState : (
        IDLE,            // Waiting for write command
        SEND,            // Trigger the write operation
        WAIT_COMPLETE    // Wait for confirmation
    );

    eWriteState : E_WriteState := E_WriteState.IDLE;

    // Modbus Write Single Register function block instance
    // From Tc2_ModbusTCP library
    fbWriteRegister : FB_MBWriteSingleRegister;

    // Configuration parameters
    sDeviceIP     : STRING(15) := '192.168.1.100';  // Target device IP
    nTCPPort      : UINT := 502;                     // Standard Modbus port
    nUnitID       : BYTE := 1;                       // Modbus unit ID
    nRegisterAddr: UINT := 100;                      // Register address to write

    // Value to write (as INT for user convenience)
    nWriteValue   : INT := 0;

    // Internal: Value converted to WORD for Modbus
    wRegisterValue : WORD;

    // Control and status
    bExecuteWrite   : BOOL;            // Trigger to start write
    bWriteDone      : BOOL;            // Write completed successfully
    bWriteError     : BOOL;            // Write failed
    nErrorID        : UDINT;           // Error code if failed
END_VAR
```

```
// State machine implementation for writing a single register
CASE eWriteState OF

    E_WriteState.IDLE:
        // Idle state - waiting for write command
        bWriteDone := FALSE;
```

```
        bWriteError := FALSE;

        IF bExecuteWrite THEN
            bExecuteWrite := FALSE;  // Reset trigger

            // IMPORTANT: Convert INT to WORD for Modbus protocol
            // Modbus registers are 16-bit unsigned values
            // Use appropriate conversion based on your data type
            wRegisterValue := INT_TO_WORD(nWriteValue);

            eWriteState := E_WriteState.SEND;
        END_IF

    E_WriteState.SEND:
        // Start the Modbus write operation
        fbWriteRegister(
            sIPAddr     := sDeviceIP,           // Target device IP address
            nTCPPort    := nTCPPort,            // TCP port (502 standard)
            nUnitID     := nUnitID,             // Modbus unit identifier
            nMBAddr     := nRegisterAddr,       // Register address to write
            nValue      := wRegisterValue,      // Value to write (WORD type!)
            bExecute    := TRUE,                // TRIGGER the operation
            tTimeout    := T#5S                 // 5 second timeout
        );

        eWriteState := E_WriteState.WAIT_COMPLETE;

    E_WriteState.WAIT_COMPLETE:
        // Wait for write operation to complete
        fbWriteRegister(bExecute := FALSE);

        IF NOT fbWriteRegister.bBusy THEN
            IF fbWriteRegister.bError THEN
                // Write failed
                bWriteError := TRUE;
                nErrorID := fbWriteRegister.nErrId;
            ELSE
                // Write successful
                bWriteDone := TRUE;
            END_IF

            eWriteState := E_WriteState.IDLE;
        END_IF

END_CASE
```

```
// Example: Writing multiple registers
// Use FB_MBWriteRegs for writing multiple consecutive registers

VAR
    fbWriteMultiple : FB_MBWriteRegs;

```

```
    // Buffer containing values to write
    // Each element is a WORD (16-bit)
    aWriteBuffer : ARRAY[0..4] OF WORD := [100, 200, 300, 400, 500];
END_VAR

// In SEND state:
fbWriteMultiple(
    sIPAddr      := '192.168.1.100',
    nTCPPort     := 502,
    nUnitID      := 1,
    nMBAddr      := 100,                      // Starting address
    nQuantity    := 5,                        // Number of registers
    cbLength     := SIZEOF(aWriteBuffer),     // Buffer size in bytes
    pSrcAddr     := ADR(aWriteBuffer),        // Pointer to source data
    bExecute     := TRUE,
    tTimeout     := T#5S
);
```

Warning: Modbus uses 16-bit unsigned WORD values for registers. When writing signed integers (INT), you must convert them using `INT_TO_WORD()`. When reading, convert back using `WORD_TO_INT()`. Failure to convert properly can result in incorrect values, especially for negative numbers.

7 Key Considerations

7.1 Virtual Machines

When testing communication between separate systems (e.g., a PLC and a simulator), ensure they have distinct IP addresses on the same network.

7.2 Error Handling

Always monitor error codes. Common error codes include:

Error Code	Hex	Description
1828	0x724	Missing TwinCAT function license
1861	0x745	Target device not responding
6	0x06	Target busy (device specific)
274	0x112	Invalid Modbus address

Table 3: Common Modbus TCP Error Codes

7.3 Port 502

This is the standard TCP port for Modbus communication. Ensure firewalls allow traffic on this port.

```
// Comprehensive error handling example
IF fbReadCoils.bError THEN
    CASE fbReadCoils.nErrId OF
        16#724:  // 1828 decimal
            sErrorMsg := 'License missing for TF6250';
```

```
        16#745:  // 1861 decimal
            sErrorMsg := 'Device not responding - check IP/connection';

        16#112:  // 274 decimal
            sErrorMsg := 'Invalid Modbus address';

    ELSE
        sErrorMsg := CONCAT('Unknown error: ', UDINT_TO_STRING(fbReadCoils.nErrId));
    END_CASE
END_IF
```

Tip: When testing Modbus communication, use a Modbus simulator (such as ModRSsim2 or Modbus Poll) on a separate machine or VM. This allows you to verify your PLC code is working correctly before connecting to actual field devices.

Quick Reference Card

TwinCAT Function Series

Series	Category
TF1xxx	System
TF2xxx	HMI
TF3xxx	Measurement
TF4xxx	Controller
TF5xxx	Motion Control
TF6xxx	Connectivity
TF7xxx	Vision
TF8xxx	Industry Specific

Modbus Function Blocks

Function Block	Purpose
FB_MBReadCoils	Read digital outputs (coils)
FB_MBReadInputs	Read digital inputs
FB_MBReadRegs	Read holding registers
FB_MBReadInputRegs	Read input registers
FB_MBWriteCoils	Write multiple coils
FB_MBWriteSingleRegister	Write single register
FB_MBWriteRegs	Write multiple registers

Asynchronous Pattern

```
// Standard pattern for all async Modbus operations
CASE eState OF
    WAIT:
        IF bTrigger THEN eState := TRIGGER; END_IF

    TRIGGER:
        fbModbus(bExecute := TRUE, ...);
        eState := BUSY;

    BUSY:
        fbModbus(bExecute := FALSE);
        IF NOT fbModbus.bBusy THEN
            // Check bError, process results
            eState := WAIT;
        END_IF
END_CASE
```

Data Type Conversions

```
// INT to WORD (for writing)
wValue := INT_TO_WORD(nIntValue);

// WORD to INT (for reading)
nIntValue := WORD_TO_INT(wValue);

// BOOL to BYTE (for coils)
byValue := BOOL_TO_BYTE(bValue);
```

Last Updated: March 14, 2026